matlabc 使用说明

这是一份**只讲怎么用**的手册：装好之后，每个功能在哪儿、怎么触发、看到什么。 不涉及内部实现，也不涉及打包发版。

你需要：VS Code 1.60 以上；一个 Python 3.6+ 解释器（扩展靠它干活，**不需要装 MATLAB**）。

---

1. 安装

1.1 装扩展

方式一：图形界面（推荐）

1. 打开 VS Code，左侧点「扩展」图标（或按 `Ctrl+Shift+X`）
2. 点扩展面板右上角的 `…` → **从 VSIX 安装...**
3. 选 `matlabc-<版本>.vsix`，装完点「重新加载」

方式二：命令行

```
code --install-extension matlabc-<版本>.vsix
```

装完请**重载窗口**：`Ctrl+Shift+P` → `Developer: Reload Window`。

1.2 让它认识你的 Python

扩展靠一个 Python 进程干活。如果 `python` 不在系统 PATH 里（或你有多个 Python）， 在设置里指明：

设置	默认值	什么时候要改
`matlabc.pythonPath`	`python`	提示「找不到 Python」、或装了多个版本
`matlabc.lspPath`	空	扩展没和你打开的工程放在一起时，填 `matlabc_lsp.py` 的绝对路径
`matlabc.enabled`	勾选	想临时关掉全部诊断与跳转
`matlabc.docLang`	`both`	内置函数说明只想看中文 / 只看英文
`matlabc.crossLanguage`	勾选	不想扫 C 源与 mex/DLL 时关掉（见第 5 节）

1.3 打开工程

用 VS Code **打开整个工程文件夹**（不是单个文件）。打开 `.m` 文件后扩展才会激活。

第一次打开会花几秒到几十秒建索引，取决于工程大小。期间跳转和关系面板可能不完整， 稍等即可，不需要重开。

---

2. 五个最常用的动作

2.1 看一个文件里有什么

按 `Ctrl+Shift+O`，列出当前文件所有函数，选中回车跳过去。

适合拿到一个陌生文件时先看骨架。

2.2 跳到定义

光标停在名字上：

按键	效果
`F12`	跳到定义
`Alt+F12`	就地预览定义（不离开当前文件）

能跳的：函数、局部函数、类方法、结构体字段（`s.field`）、结构体变量（`s`）、`global` 声明。 字段会落到**首次赋值**那一行 —— 因为字段没有"定义语句"。



示意图：函数上方那行「3 个调用者 · 复杂度 4」是本扩展加的（CodeLens）； `F12` 跳过去，`Alt+F12` 就地预览。

2.3 看它和谁有关系

光标停在任意名字上，右侧「关系」面板会显示：

- 它被谁调用（上游）
- 它调用了谁（下游）
- 如果是变量：在哪里声明、在哪里出现

- 如果是结构体字段：哪些位置写入、哪些位置读取

面板里点开的分组会**被记住**：光标移到别处再移回来，仍然是你上次展开的样子。



示意图：左边是代码，右边是上下游。落不到 MATLAB 里的调用会标成跨语言， 并给出 C 文件与行号。

2.4 改代码前先看影响面

关系面板里的「上游」就是影响面；想看完整调用链，把光标放在目标函数上， 面板会列出**从入口到它的每一条来路**（带文件名与行号）。

→ 改一个函数之前，先看这里：谁在调它，就有谁会因为你的改动而出问题。

2.5 生成一份可转发的报告

命令面板（`Ctrl+Shift+P`）→ `matlabc: 生成 HTML 分析报告`。

浏览器会打开一份单文件报告：工程概览、热点函数、耦合关系、诊断清单。 可以直接发给同事，或打印成 PDF。

---

3. 命令清单

按 `Ctrl+Shift+P`，输入 `matlabc`，共 10 条：



示意图：命令面板里输入 `matlabc`。

想先看哪里最值得改，用「热点函数排行」：



示意图：按「复杂度 × 调用者数」排序，选中即跳转。

命令	作用
`matlabc: 打开使用说明`	就是本文（随扩展一起分发，离线可看）
`matlabc: 打开使用说明（PDF）`	本文的 PDF 版：四张示意图直接印在页面上，也适合转发
`matlabc: 打开使用导览`	8 步动画，一屏讲清每个功能在哪 —— **第一次用建议先看它**
`matlabc: 热点函数排行`	全工程按「复杂度 × 调用者数」排序，选中即跳转
`matlabc: 热点函数排行（当前目录）`	只看当前目录 —— 跨模块的分数没有可比性
`matlabc: 预览调用关系图`	当前函数的关系图（纵向）
`matlabc: 预览调用关系图（横向）`	同上，横向布局，宽屏更好读
`matlabc: 查看名字的关系（函数/变量）`	用命令的方式打开右侧「关系」面板
`matlabc: 生成 HTML 分析报告`	生成整份报告并在浏览器打开
`matlabc: 生成 MCP 配置（给 AI 客户端）`	见第 6 节

---

4. 编辑器里会看到什么

打开一个 `.m` 文件后，除了上面那些主动操作，还有些东西是**自动出现**的：

你会看到	在哪	意思
函数名上方一行小字	每个函数的第一行上方	这个函数有多少调用者、被调用多少次、圈复杂度
波浪线	代码里	诊断：未初始化的变量、未使用的参数、可疑的隐式转换、缺文档等
不同颜色	光标所在处	参数、局部变量、函数、字段、全局变量、内置函数 分别着色
顶部弹出的函数说明	光标停在函数名上	内置函数给中文说明；你自己的函数给签名与位置

这些不需要任何操作，保存文件后会自动更新。

---

5. 跨语言：MATLAB 调不到的东西

当一个调用在 MATLAB 里**解析不到**时（`mex`、`calllib`、预编译库）， 关系面板会多出几组：

分组	意思
`⇢ 跨语言（C 源）`	工程里有这个函数的 `.c/.cpp` 定义，直接给文件名与行号
`⇢ 二进制（mex / 库）`	同名 `.mexw64` / `.dll` / `.so` 就在工程里，入口名写在文件名上
`⇢ calllib 调用`	`calllib('lib','fn')` 里的 `fn` 是否真在该库的导出表里（✓ / ✗ / ?）
`⇢ 仍连不上`	C 源与工程内二进制都没有 —— 多半来自工具箱或动态调用

没有看到这些分组? 说明工程里确实没找到 `.c` / `.mexw64` / `.dll` / `.so`； 如果你确认有，检查设置 `matlabc.crossLanguage` 是否被关掉了 —— 关掉时那里会**明确写「已关闭」**，而不是安静地不显示。

---

6. 让 AI 助手读你的工程（可选）

命令面板 → `matlabc: 生成 MCP 配置（给 AI 客户端）`，按提示把生成的 JSON 贴进 你的 AI 客户端配置（Claude Desktop / Cursor / CodeBuddy 等），重启客户端即可。

6.1 AI 能查什么

接入后它一共能调用 **21 个工具**，常用的几类：

想问什么	AI 会调用	你会拿到
这工程多大	`matlabc_stats`	文件数、函数数、调用边、最复杂的几个
这个文件里有什么	`matlabc_outline`	函数清单与行号
谁调用了它 / 它调用了谁	`matlabc_impact`	完整来路、上游下游、解析不到的调用
改这几个文件谁要回归	`matlabc_changed`	需要先验证的入口
这个名字在哪	`matlabc_search` / `matlabc_definition` / `matlabc_references`	位置与引用
字段在哪读、在哪写	`matlabc_fields`	按变量名分组的读写位置
工程有什么毛病	`matlabc_diagnostics`	未初始化、未使用等
这个 MATLAB 函数调的 C 在哪	`matlabc_cbridge`	C 文件与行号；反向也能查

6.2 AI 改代码：带验收、可回滚

这一点和别的助手不一样，值得单独说。

普通的"让 AI 改代码"是：它改完，你自己看对不对。 这里改代码走的是**任务**，每次都自带验收条件：

任务	做什么	验收条件
`rename`	把一个函数名改掉	旧名字归零 · 函数数不变 · 调用边不变 · 诊断不增加
`rename-field`	改结构体字段（`s.old` → `s.new`）	同上；同名变量与字符串里的名字**不动**
`extract`	把一段代码提成函数	函数数 +1 · 诊断不增加 · 结构配平
`move`	把整个函数搬到独立文件	函数总数不变 · 诊断不增加 · 定义真的迁移了

流程固定是两步：

```
1. 先"算一遍"（不写盘）— 工具名 matlabc_task_plan
   改哪些位置、验收条件是什么；你可以先看再决定
2. 再"执行" — 工具名 matlabc_task_apply
   写入 → 重新分析 → 逐条核对 → 任何一条不过就**逐字节回滚**
```

所以它能做到：改坏了会自己退回去，而不是留一个半成品给你发现。

> 两个边界要说清楚： - 验收条件管的是**结构**（函数数、调用边、诊断、名字残留）， 不是"业务逻辑正确"——语义对不对仍然要你判断； - `extract` 的接口如果你让工具自己推断，那是启发式，**先看它的推断结果再执行**。

6.3 它不会做什么

- 不在你不知情时改文件：所有写操作都是「先算 → 你确认 → 再落盘」
- 不覆盖已有文件：`extract --to` / `move` 遇到同名目标会直接拒绝
- 不改工程外的东西：所有路径都必须在工程目录内

---

7. 常见问题

Q：打开工程后什么反应都没有？ 确认你打开的是**文件夹**、并且当前文件是 `.m`。扩展只在 MATLAB 文件上激活。

Q：提示找不到 Python / 启动失败？ 设置 `matlabc.pythonPath` 为 Python 的绝对路径（例如 `C:\Python310\python.exe`）， 然后重载窗口。命令行先用它跑一次 `python matlabc_lsp.py --help` 确认能起来。

Q：大工程很慢、编辑卡顿？ 第一次分析最慢，之后增量更新。实在太大可以关掉 `matlabc.crossLanguage`（省掉扫 C 源与二进制）。

Q：诊断太多，满屏波浪线？ `Ctrl+Shift+P` → `matlabc.enabled` 临时关掉，或只在你关心的目录里工作。 诊断按严重度排序，先处理「错误」级别。

Q：让 AI 改代码，改坏了怎么办？ 它改代码走的是任务：执行前会先把验收条件列出来，改完重新分析逐条核对， **任何一条不过就自动回滚**，文件恢复到改前逐字节相同。所以最坏情况是"没改成"， 而不是"改成半截"。

Q：AI 改完我想自己再确认一遍？ 看它返回的 `verdict`： `pass` 才是全部验收通过； `rollback` 表示它自己退回去了 （返回里会写哪条判据没过）。另外每个任务在动手前都能先"算一遍"不写盘， 那份计划里就是将要改的全部位置。

Q：误报了怎么办？ 把那个函数名和文件发给我 —— 误报比漏报更值得修。

Q：想回退到旧版本？ 重新装旧版 VSIX 即可（`code --install-extension matlabc-<旧版本>.vsix`），设置与缓存都在工程目录的 `.matlabc/` 下，回退后仍然可用。

---

8. 一分钟上手

1. 装扩展 → 重载
2. 打开工程文件夹 → 等索引进度过去
3. `Ctrl+Shift+P` → `matlabc: 打开使用导览`，看一遍 8 步
4. 随便点进一个 `.m`，光标停在一个函数名上，看右侧「关系」面板
5. 按 `F12` 试试能不能跳过去